Chapter 4 — Aggregation: Summarising Many Rows

Chapters 2 and 3 worked **row by row**. Most business questions — "how many orders shipped last month?", "average salary per department?", "which customers spent over 1,000?" — squeeze **many rows into one answer**. That is **aggregation**.

Sections: 4.1 aggregate functions (and their `NULL`/empty-input rules) · 4.2 `GROUP BY` · 4.3 `HAVING` · 4.4 conditional aggregation (`FILTER`, `CASE`) and pivot-style reports · 4.5 the logical query processing order · 4.6 subtotals with `GROUPING SETS`, `ROLLUP`, `CUBE`.

Running the examples. All queries are in `code/`, run on the book's PostgreSQL 14 server (`bash tools_run_sql.sh sql '04-*'` runs this chapter on fresh databases); `(null)` is psql's `\pset null` display. Data: the `company` database (chapter 1, 1.11; `code/00_schema.sql`). Quirks that matter here: Asha (CEO) and Rohan (intern) have **no department**; Research has **no** employees; only 3 people have a commission; two salaries are shared by two people each; `sales_rep_id` is `NULL` for online orders; one product has no category; two customers never ordered.

4.1 Aggregate functions

What it is

An **aggregate function** takes a *set of rows* and returns *one value* summarising them — like a till's end-of-day count of sales, total taken and largest sale.

Function	Returns	Works on	Notes
`COUNT(*)`	number of **rows**	any rows	Counts rows, even rows that are all `NULL`.
`COUNT(expr)`	number of rows where `expr` is **not NULL**	any type	The usual way to count "how many have a value".
`COUNT(DISTINCT expr)`	number of **different** non-NULL values	any type	Duplicates and `NULL`s are dropped first.
`SUM(expr)`	total	numbers, intervals (durations)	`NULL` if there are no non-NULL inputs.
`AVG(expr)`	arithmetic mean = `SUM / COUNT(expr)`	numbers, intervals	Divides by the count of **non-NULL** values.
`MIN(expr)` / `MAX(expr)`	smallest / largest	anything that can be sorted: numbers, text, dates	Text uses the column's collation (alphabetical order).

A **scalar function** (chapter 3) returns one value *per row*; an aggregate returns one value *per group*. Without `GROUP BY`, the whole table (after `WHERE`) is one group, so the query returns exactly one row.

Why it matters

Reports, dashboards, KPIs (key performance indicators — a business's headline numbers), billing, fraud checks and most SQL interview questions are aggregations. Summarising inside the database is far cheaper than shipping every row to the application: it reads rows where they live and returns one small result.

How it works

1. Build the input rows (`FROM` and any joins — a **join** combines rows of several tables by matching values, Chapter 5); drop rows failing `WHERE`.
2. Per aggregate, walk the rows keeping a small running **state** (counter, running total, smallest so far…), **skipping** `NULL`**s** — every aggregate except `COUNT(*)` does.
3. Turn the state into the result (for `AVG`, total ÷ count).

```
SELECT count(*)              AS employees,
       sum(salary)           AS payroll,
       round(avg(salary), 2) AS avg_salary,
       min(salary)           AS lowest,
       max(salary)           AS highest
FROM employees;
```

Output:

```
 employees |  payroll   | avg_salary |  lowest  |  highest
-----------+------------+------------+----------+-----------
        15 | 1617000.00 |  107800.00 | 30000.00 | 250000.00
(1 row)
```

The three flavours of COUNT

```
SELECT count(*)                AS all_rows,
       count(email)            AS with_email,
       count(dept_id)          AS with_dept,
       count(DISTINCT dept_id) AS distinct_depts,
       count(DISTINCT job_title) AS distinct_titles
FROM employees;
```

Output:

```
 all_rows | with_email | with_dept | distinct_depts | distinct_titles
----------+------------+-----------+----------------+-----------------
       15 |         13 |        13 |              5 |              12
(1 row)
```

`count(*)` = 15 rows; `count(email)` = 13 (two have no email); `count(dept_id)` = 13 people with a department, spread over `count(DISTINCT dept_id)` = 5 departments — the two `NULL`s are not a sixth "value".

NULLs are skipped — and that changes averages

```
SELECT count(commission_pct)            AS non_null,
       avg(commission_pct)              AS avg_ignoring_nulls,
       avg(coalesce(commission_pct, 0)) AS avg_nulls_as_zero
FROM employees;
```

Output:

```
 non_null |   avg_ignoring_nulls   |   avg_nulls_as_zero
----------+------------------------+------------------------
        3 | 0.08333333333333333333 | 0.01666666666666666667
(1 row)
```

`AVG` divided by **3** (people with a commission), not 15. "Average rate *among people who earn commission*" → 0.083 is right; "across all staff, no commission = zero" → say so with `COALESCE(commission_pct, 0)`, giving 0.017. Neither is wrong; the mistake is not knowing which you asked.

Empty input

```
SELECT count(*) AS n, sum(salary) AS total, avg(salary) AS mean, max(salary) AS top
FROM employees WHERE salary > 1000000;
```

Output:

```
 n | total  |  mean  |  top
---+--------+--------+--------
 0 | (null) | (null) | (null)
(1 row)
```

With zero input rows `COUNT` answers `0`; `SUM`, `AVG`, `MIN`, `MAX` answer `NULL` ("no total of nothing"). You still get **one row** — an aggregate query without `GROUP BY` always returns exactly one. For `0` from an empty sum write `COALESCE(SUM(x), 0)`.

Other useful points

`MIN` / `MAX` work on any sortable type, and an aggregate can take an **expression**, not just a column:

```
SELECT min(hire_date) AS first_hire, max(hire_date) AS latest_hire,
       min(last_name) AS first_alpha, max(last_name) AS last_alpha
FROM employees;

SELECT round(sum(quantity * unit_price * (1 - discount)), 2) AS gross_revenue
FROM order_items;
```

Output:

```
 first_hire | latest_hire | first_alpha | last_alpha
------------+-------------+-------------+------------
 2015-01-05 | 2025-06-02  | Brooks      | Wei
(1 row)

 gross_revenue
---------------
      10026.61
(1 row)
```

A `WHERE` clause simply shrinks the set being aggregated (`WHERE status = 'delivered'` → 7 orders, first 2025-01-03, last 2025-04-09).

Beyond the big five (PostgreSQL)

The extra PostgreSQL aggregates you will actually use:

```
SELECT string_agg(first_name, ', ' ORDER BY first_name) AS engineers,
       array_agg(emp_id ORDER BY emp_id)                AS ids,
       bool_and(salary > 90000)                         AS all_above_90k,
       bool_or(email IS NULL)                           AS any_missing_email
FROM employees WHERE dept_id = 1;

SELECT round(stddev_samp(salary), 2) AS stddev,
       percentile_cont(0.5) WITHIN GROUP (ORDER BY salary) AS median,
       mode() WITHIN GROUP (ORDER BY salary)               AS most_common
FROM employees;
```

Output:

```
         engineers          |    ids    | all_above_90k | any_missing_email
----------------------------+-----------+---------------+-------------------
 Meera, Neha, Rahul, Vikram | {2,3,4,5} | t             | t
(1 row)

  stddev  | median | most_common
----------+--------+-------------
 57217.88 |  98000 |    68000.00
(1 row)
```

Aggregate	What it gives	Typical use
`string_agg(x, sep ORDER BY …)`	all values joined into one string	"list the products in each order"
`array_agg(x ORDER BY …)`	all values as an array (an ordered list stored as one value)	feed an app, or `unnest` (expand back into rows) later
`bool_and` / `bool_or`	TRUE if all / any inputs are true	"did every item ship?"
`stddev_samp`, `variance`	spread of the values (standard deviation ≈ how far values typically sit from the average)	analytics
`percentile_cont(f) WITHIN GROUP (ORDER BY x)`	interpolated percentile (0.5 = median)	median salary, p95 latency (the time 95 % of requests finish within)
`percentile_disc(f) …`	percentile that is an actual input value	"the median order", not an average of two
`mode() WITHIN GROUP (ORDER BY x)`	most frequent value	most common salary / status

> **Key term — percentile and median:** the *f*-th percentile is the value below which a fraction *f* of sorted values fall; the **median** is the 50th percentile, which (unlike the average) is not dragged by a few huge values. *Interpolated* = blended from the two values either side of the cut, so it may never occur in the data. **Latency** is how long one request takes; "p95 latency" is its 95th percentile (System Design Fundamentals).

`percentile_cont`, `percentile_disc` and `mode` are **ordered-set aggregates**: `WITHIN GROUP (ORDER BY …)` says which order to read the values in.

Dialect differences

Feature	PostgreSQL	MySQL 8	SQLite	SQL Server
Join values into a string	`string_agg(x, ',')`	`GROUP_CONCAT(x SEPARATOR ',')`	`group_concat(x, ',')`	`STRING_AGG(x, ',')` (2017+)
`AVG` of an integer column	exact `numeric`	decimal	floating point	**integer** (truncated!) — cast first: `AVG(1.0 * x)`
Median	`percentile_cont(0.5) WITHIN GROUP (…)`	no built-in	no built-in	`PERCENTILE_CONT(0.5) WITHIN GROUP (…) OVER ()` (window form, Chapter 8)
Boolean aggregates	`bool_and`, `bool_or`	`MIN` / `MAX` of 0/1	`min` / `max` of 0/1	`MIN` / `MAX` of a `CASE`

Common mistakes

- **`COUNT(column)` vs `COUNT(*)` mixed up** — they differ by the number of `NULL`s in that column.
- **Averaging with NULLs without thinking** (commission example).
- **`SUM` of an empty set used as a number** in application code — it is `NULL`, not `0`.
- **Integer averages in SQL Server** (`AVG` of `int` returns an `int`).
- **`COUNT(DISTINCT a, b)`** — PostgreSQL does not accept two arguments; count distinct *rows* instead: `COUNT(DISTINCT (a, b))` or a subquery with `SELECT DISTINCT a, b`.

In the interview

> **30-second answer.** "Aggregate functions reduce a set of rows to one value: `COUNT`, `SUM`, `AVG`, `MIN`, `MAX`. All of them ignore `NULL` inputs except `COUNT(*)`, which counts rows. On empty input `COUNT` returns 0 and the others return `NULL`. Without `GROUP BY` the whole filtered table is one group, so you get one row."

- *Q: What is the difference between `COUNT(*)`, `COUNT(1)` and `COUNT(col)`?* — `COUNT(*)` and `COUNT(1)` both count rows (1 is never NULL), and PostgreSQL treats them the same way; `COUNT(col)` counts non-NULL values of `col`.
- *Q: `AVG(x)` vs `SUM(x)/COUNT(*)`?* — They differ when `x` has NULLs: `AVG` divides by `COUNT(x)`. Also `SUM(int)/COUNT(*)` is integer division in PostgreSQL.
- *Q: How do you compute a median in PostgreSQL?* — `percentile_cont(0.5) WITHIN GROUP (ORDER BY x)`.

Self-check

1. Values 10, NULL, 20: `COUNT(*)`, `COUNT(col)`, `SUM(col)`, `AVG(col)`? → 3, 2, 30, 15.
2. `SELECT MAX(salary) FROM employees WHERE 1 = 0` — zero rows or one? → One row containing `NULL`.
3. Count employees with *no* email? → `COUNT(*) - COUNT(email)`, or `COUNT(*) FILTER (WHERE email IS NULL)` (§4.4).

4.2 GROUP BY

What it is

`GROUP BY` splits rows into **groups** — one per distinct value (or combination) of the listed expressions — and runs the aggregates **once per group**, giving **one row per group** (like sorting exam papers into piles by class and writing count and average mark on each pile).

How it works

1. Build and filter the rows (`FROM`, `WHERE`).
2. Put rows with the **same values** in all `GROUP BY` expressions into the same group. For grouping, all `NULL`s count as equal, so every row with a `NULL` key lands in **one** `NULL` group.
3. For each group, compute each aggregate over just that group's rows.
4. Output one row per group. The `SELECT` list may contain only the grouping expressions, aggregates, constants, or expressions built from those.

```
SELECT dept_id, count(*) AS headcount, sum(salary) AS payroll
FROM employees
GROUP BY dept_id
ORDER BY dept_id;
```

Output:

```
 dept_id | headcount |  payroll
---------+-----------+-----------
       1 |         4 | 523000.00
       2 |         4 | 358000.00
       3 |         2 | 181000.00
       4 |         2 | 211000.00
       5 |         1 |  64000.00
  (null) |         2 | 280000.00
(6 rows)
```

Five departments plus a `NULL` **group** (Asha and Rohan). Research does not appear: groups come only from existing rows. Departments with zero employees need an outer join (keeps rows with no match; Chapter 5).

Grouping by several columns

With two grouping columns, each **distinct pair** is a group:

```
SELECT dept_id, job_title, count(*) AS n, round(avg(salary)) AS avg_salary
FROM employees
WHERE dept_id IN (1, 2)
GROUP BY dept_id, job_title
ORDER BY dept_id, job_title;
```

Output:

```
 dept_id |      job_title      | n | avg_salary
---------+---------------------+---+------------
       1 | Engineer            | 2 |      98000
       1 | Engineering Manager | 1 |     185000
       1 | Senior Engineer     | 1 |     142000
       2 | Sales Manager       | 1 |     150000
       2 | Sales Rep           | 3 |      69333
(5 rows)
```

`GROUP BY a, b` and `GROUP BY b, a` give the same groups; output order comes only from `ORDER BY`.

Grouping by an expression

Group by a computed value, such as a timestamp's month:

```
SELECT date_trunc('month', placed_at)::date AS month, count(*) AS orders
FROM orders
GROUP BY date_trunc('month', placed_at)
ORDER BY month;
```

Output:

```
   month    | orders
------------+--------
 2025-01-01 |      3
 2025-02-01 |      2
 2025-03-01 |      2
 2025-04-01 |      3
 2025-05-01 |      2
 2025-06-01 |      2
(6 rows)
```

PostgreSQL also accepts a `SELECT` alias or column position in `GROUP BY` (non-standard; if an alias clashes with a real column name, the real column wins):

```
SELECT extract(year FROM hire_date) AS hire_year, count(*) AS hires
FROM employees
GROUP BY hire_year
ORDER BY 1;
```

Output:

```
 hire_year | hires
-----------+-------
      2015 |     1
      2016 |     1
 ...
      2025 |     1
(11 rows)
```

Grouping usually follows a join (chapter 5) — revenue per category:

```sql
SELECT p.category, sum(oi.quantity) AS units,
       round(sum(oi.quantity * oi.unit_price * (1 - oi.discount)), 2) AS revenue
FROM order_items oi JOIN products p ON p.product_id = oi.product_id
GROUP BY p.category
ORDER BY revenue DESC;
```

Output:

```
  category   | units | revenue
-------------+-------+---------
 Electronics |    16 | 7314.79
 Furniture   |     5 | 1940.47
 Accessories |    11 |  671.35
 (null)      |     2 |  100.00
(4 rows)
```

A `GROUP BY` with no aggregates returns each distinct group once: `SELECT status FROM orders GROUP BY status` = `SELECT DISTINCT status FROM orders` (4 rows).

The single-value rule

Each output row stands for a whole group, so a column neither grouped nor aggregated could have **several values** per group. SQL refuses to guess:

```sql
SELECT dept_id, first_name, count(*) FROM employees GROUP BY dept_id;
```

Output:

```
ERROR:  column "employees.first_name" must appear in the GROUP BY clause or be used in an aggregate function
LINE 1: SELECT dept_id, first_name, count(*) FROM employees GROUP BY...
                        ^
```

Exception: grouping by a table's **primary key** (the column uniquely identifying each row) means every other column of that table has one value per group (it is *functionally dependent* on the key — Chapter 11), so PostgreSQL (9.1+) allows it:

```sql
SELECT d.dept_id, d.name, count(e.emp_id) AS headcount
FROM departments d JOIN employees e ON e.dept_id = d.dept_id
GROUP BY d.dept_id
ORDER BY d.dept_id;
```

Output:

```
 dept_id |    name     | headcount
---------+-------------+-----------
       1 | Engineering |         4
       2 | Sales       |         4
       3 | Marketing   |         2
       4 | Finance     |         2
       5 | Support     |         1
(5 rows)
```

Database	Non-grouped, non-aggregated column in SELECT
PostgreSQL	Error, unless the column depends on a grouped primary key.
MySQL 8	Error by default (`ONLY_FULL_GROUP_BY` mode, on since 5.7.5); also detects primary-key dependency. With the mode off it returns an arbitrary value.
SQLite	Allowed ("bare column"); the value comes from an arbitrary row of the group (or from the row holding the `min` / `max` if the query has exactly one of those).
SQL Server, Oracle	Always an error.

Trade-offs and performance

Two physical strategies (shown by `EXPLAIN`, which prints the query plan — Chapter 12):

Strategy	How	Trade-off
Hash aggregation	in-memory hash table (turns each key into a slot number, ~one-step lookup) keyed by group values	fast, no sort; needs memory for all groups
Sort + group aggregation	sort by the grouping key, summarise each run of equal keys	can use an index (sorted lookup structure beside the table) already in that order; streams with little memory

Grouping by high-cardinality columns (many distinct values) creates many groups and costs more memory.

When not to use it

- Each **row** *and* its group total (salary next to department average)? `GROUP BY` collapses the rows; use a **window function** (computes over related rows but keeps every row): `avg(salary) OVER (PARTITION BY dept_id)` — Chapter 8.
- Only distinct values? `SELECT DISTINCT` states the intent more clearly.

Common mistakes

- "Fixing" the non-grouped-column error by adding the column to `GROUP BY`, silently creating *smaller* groups. Ask "one row per *what*?" and group by exactly that.
- Expecting groups for values with no rows (Research).
- Forgetting `NULL` keys form their own group — label with `COALESCE(dept_id::text, 'none')`.
- Relying on `GROUP BY` to sort — hash aggregation returns groups in no order; add `ORDER BY`.

In the interview

> **30-second answer.** "`GROUP BY` partitions the filtered rows by the grouping columns and computes the aggregates once per partition, returning one row per group. Every selected column must be grouped or aggregated — unless it depends on a grouped primary key. `NULL`s form one group. For per-row output with group totals, use window functions instead."

- *Q: Does `GROUP BY` guarantee sorted output?* — No. Only `ORDER BY` does.
- *Q: `GROUP BY` vs `DISTINCT`?* — Without aggregates they give the same rows; `GROUP BY` is needed when you want to compute something per group.
- *Q: How do you find duplicate emails?* — `SELECT lower(email), count(*) FROM customers GROUP BY lower(email) HAVING count(*) > 1;` (the `HAVING` part is next).

Self-check

1. `GROUP BY dept_id` gives 6 rows and there are 6 departments. Is Research one? → No; the sixth is the `NULL` group.
2. Does `GROUP BY a, b` give more, fewer or the same groups as `GROUP BY a`? → The same or more — each `a` group is split by `b`.

4.3 HAVING

What it is

`HAVING` filters **groups**, **after** aggregation, so it can use aggregates (`count(*) > 2`). `WHERE` filters **rows** *before* grouping, so it cannot.

	WHERE	HAVING
Filters	individual rows	whole groups
Runs	before `GROUP BY`	after `GROUP BY` and aggregation
Can use aggregates?	**No**	Yes
Can use plain columns?	Yes	Only grouping columns (or inside aggregates)
Effect on work	fewer rows to group → cheaper	groups are already computed

How it works

```sql
SELECT dept_id, count(*) AS headcount
FROM employees
GROUP BY dept_id
HAVING count(*) > 2
ORDER BY dept_id;
```

Output:

```
 dept_id | headcount
---------+-----------
       1 |         4
       2 |         4
(2 rows)
```

Group 15 employees into 6 groups, count each, keep groups over 2 — Engineering and Sales.

`WHERE` removes *rows*, then `HAVING` removes *groups*. "Among non-cancelled orders, which customers ordered at least twice?"

```sql
SELECT customer_id, count(*) AS orders
FROM orders
WHERE status <> 'cancelled'
GROUP BY customer_id
HAVING count(*) >= 2
ORDER BY customer_id;
```

Output:

```
 customer_id | orders
-------------+--------
           1 |      3
           2 |      2
           5 |      2
           6 |      2
(4 rows)
```

Customer 3 (Chen) has two orders but one is cancelled: `WHERE` drops it, the group has 1 row, `HAVING` drops the group.

`HAVING` may use an aggregate not in the `SELECT` list, and combine conditions:

```sql
SELECT dept_id
FROM employees
GROUP BY dept_id
HAVING max(salary) > 140000
ORDER BY dept_id;

SELECT dept_id, count(*) AS n, round(avg(salary)) AS avg_salary
FROM employees
WHERE dept_id IS NOT NULL
GROUP BY dept_id
HAVING avg(salary) > 90000 AND count(*) >= 2
ORDER BY avg_salary DESC;
```

Output:

```
 dept_id
---------
       1
       2
  (null)
(3 rows)

 dept_id | n | avg_salary
---------+---+------------
       1 | 4 |     130750
       4 | 2 |     105500
       3 | 2 |      90500
(3 rows)
```

The `NULL` group qualifies in the first result because it holds the CEO's salary.

An aggregate in WHERE is an error

```
SELECT dept_id, count(*) FROM employees WHERE count(*) > 2 GROUP BY dept_id;
```

Output:

```
ERROR:  aggregate functions are not allowed in WHERE
LINE 1: SELECT dept_id, count(*) FROM employees WHERE count(*) > 2 G...
                                                      ^
```

When `WHERE` runs, groups don't exist yet (§4.5).

Row conditions belong in WHERE

A condition on a grouping column gives the same answer in either place: `… GROUP BY dept_id HAVING dept_id = 2` and `… WHERE dept_id = 2 GROUP BY dept_id` both return `2 | 4`. Prefer `WHERE`: it removes rows *before* the expensive grouping, can use an index, and reads as a row filter. (PostgreSQL's planner — the component deciding how to execute a query — often pushes such a `HAVING` down into `WHERE`, but not every database does.)

HAVING without GROUP BY

`HAVING` without `GROUP BY` treats the whole input as **one group**: one row if the condition holds, none otherwise:

```
SELECT count(*) AS products FROM products HAVING count(*) > 5;
SELECT count(*) AS products FROM products HAVING count(*) > 50;
```

Output:

```
 products
----------
        9
(1 row)

 products
----------
(0 rows)
```

Common mistakes

- Aggregate condition in `WHERE` (error) or row condition in `HAVING` (works, slower, confusing).
- `SELECT` alias in `HAVING`: `HAVING headcount > 2` fails in PostgreSQL (`column "headcount" does not exist`) — `HAVING` runs before `SELECT` names outputs; repeat the aggregate. (MySQL allows the alias.)
- Forgetting `WHERE` changes what `HAVING` sees (Chen).

In the interview

> **30-second answer.** "`WHERE` filters rows before grouping and can't use aggregates; `HAVING` filters groups after aggregation and can. Use `WHERE` for anything that doesn't need an aggregate, because it shrinks the input early."

- *Q: Find departments with more than 3 employees.* — `SELECT dept_id FROM employees GROUP BY dept_id HAVING COUNT(*) > 3;`
- *Q: Can you use `HAVING` without `GROUP BY`?* — Yes; the whole table is one group, so you get one row or none.
- *Q: Find duplicate rows.* — `GROUP BY` all the columns that define "duplicate" and `HAVING COUNT(*) > 1`.

Self-check

1. Rewrite `HAVING dept_id <> 5` in the better place. → `WHERE dept_id <> 5` — it also drops `NULL` rows, just as the `HAVING` version drops the `NULL` group.
2. `SELECT count(*) FROM t HAVING count(*) > 0` on an empty table? → No rows (count 0 filters out the one group).

4.4 Conditional aggregation: FILTER and CASE

What it is

Conditional aggregation aggregates only rows meeting a condition, *inside* one query, so several differently-filtered numbers sit side by side. Standard clause: `aggregate(...) FILTER (WHERE condition)` (PostgreSQL 9.4+, SQLite 3.30+); every database supports a `CASE` inside the aggregate.

The problem it solves

"Per customer: total, delivered, cancelled, online orders." Plain `WHERE` needs four queries (or subqueries — queries nested inside another), each scanning `orders` again; conditional aggregation does it in **one pass**.

How it works

`FILTER (WHERE cond)` feeds only matching rows into that one aggregate; other aggregates still see all rows of the group.

```sql
SELECT customer_id,
       count(*)                                        AS total,
       count(*) FILTER (WHERE status = 'delivered')    AS delivered,
       count(*) FILTER (WHERE status = 'cancelled')    AS cancelled,
       count(*) FILTER (WHERE sales_rep_id IS NULL)    AS online
FROM orders
GROUP BY customer_id
ORDER BY customer_id;
```

Output:

```
 customer_id | total | delivered | cancelled | online
-------------+-------+-----------+-----------+--------
           1 |     3 |         2 |         0 |      1
           2 |     2 |         2 |         0 |      1
           3 |     2 |         1 |         1 |      1
           4 |     1 |         0 |         1 |      0
           5 |     2 |         1 |         0 |      1
           6 |     2 |         0 |         0 |      1
           7 |     1 |         0 |         0 |      0
           8 |     1 |         1 |         0 |      1
(8 rows)
```

The portable `CASE` versions give identical numbers (`code/04_conditional_aggregation.sql`):

- `SUM(CASE WHEN cond THEN 1 ELSE 0 END)` — adds 1 per matching row.
- `COUNT(CASE WHEN cond THEN 1 END)` — non-matching rows give `NULL`, which `COUNT` skips.

Ratios and percentages

"What share of X is Y": divide a filtered aggregate by the unfiltered one, multiplying by `100.0` (not `100`) to avoid integer division.

```
SELECT dept_id,
       sum(salary) AS payroll,
       round(100.0 * sum(salary) FILTER (WHERE hire_date < '2020-01-01') / sum(salary), 1) AS pct_veterans
FROM employees
WHERE dept_id IS NOT NULL
GROUP BY dept_id
ORDER BY dept_id;
```

Output:

```
 dept_id |  payroll  | pct_veterans
---------+-----------+--------------
       1 | 523000.00 |         62.5
       2 | 358000.00 |         41.9
       3 | 181000.00 |         66.3
       4 | 211000.00 |         64.0
       5 |  64000.00 |       (null)
(5 rows)
```

Support shows `NULL`, not 0: its only employee was hired in 2023, so the filtered `SUM` saw no rows. Use `COALESCE(…, 0)` to show 0.

Pivot-style reports

A **pivot** turns rows into columns (one column per category). Conditional aggregation is the standard SQL pivot when the columns are known in advance:

```
SELECT p.category,
       sum(oi.quantity) FILTER (WHERE o.status = 'delivered') AS delivered_units,
       sum(oi.quantity) FILTER (WHERE o.status = 'shipped')   AS shipped_units,
       sum(oi.quantity) FILTER (WHERE o.status = 'pending')   AS pending_units
FROM order_items oi
JOIN orders o   ON o.order_id = oi.order_id
JOIN products p ON p.product_id = oi.product_id
GROUP BY p.category
ORDER BY p.category NULLS LAST;
```

Output:

```
  category   | delivered_units | shipped_units | pending_units
-------------+-----------------+---------------+---------------
 Accessories |               6 |        (null) |             5
 Electronics |               9 |             3 |             1
 Furniture   |               2 |             3 |        (null)
 (null)      |               2 |        (null) |        (null)
(4 rows)
```

Dynamic pivots (columns unknown until you see the data) and `crosstab` (a PostgreSQL extension function for pivots): Chapter 14.

Trade-offs

Approach	Portable?	Readability	Notes
`agg(x) FILTER (WHERE c)`	PostgreSQL, SQLite 3.30+ (not MySQL, SQL Server)	Best — the condition is separate from the value	Works with every aggregate, including `string_agg`, `array_agg`.
`SUM(CASE WHEN c THEN 1 ELSE 0 END)`	Everywhere	Good	Returns 0, not `NULL`, when nothing matches.
`COUNT(CASE WHEN c THEN 1 END)`	Everywhere	Good	Relies on `COUNT` skipping `NULL`.
Separate queries / subqueries	Everywhere	Verbose	Several scans of the table.

MySQL booleans are 0/1, so `SUM(status = 'delivered')` works there; PostgreSQL can't sum `boolean` — write `SUM((status = 'delivered')::int)`.

Common mistakes

- `COUNT(CASE WHEN c THEN 1 ELSE 0 END)` counts *every* row (0 is not `NULL`). Use `SUM` with `ELSE 0`, or `COUNT` with no `ELSE`.
- Integer division in ratios (`100 * a / b`).
- Putting the condition in `WHERE` when other columns need unfiltered rows — it filters *all* aggregates.

In the interview

> **30-second answer.** "Conditional aggregation computes several filtered aggregates in one pass: `COUNT(*) FILTER (WHERE status='x')` in PostgreSQL, or `SUM(CASE WHEN status='x' THEN 1 ELSE 0 END)` anywhere. It's how you do per-group breakdowns, ratios and fixed pivots."

- *Q: Share of cancelled orders per month?* — `GROUP BY` month, `ROUND(100.0 * COUNT(*) FILTER (WHERE status='cancelled') / COUNT(*), 1)`.
- *Q: Why does `COUNT(CASE … ELSE 0 END)` give the wrong number?* — Because `COUNT` counts non-NULL values and 0 is non-NULL.

Self-check

1. "Employees with no email per department", two ways? → `COUNT(*) FILTER (WHERE email IS NULL)`; `SUM(CASE WHEN email IS NULL THEN 1 ELSE 0 END)`.
2. A `FILTER`ed `SUM` where no row matches? → `NULL`.

4.5 The logical order of query processing

What it is

You **write** `SELECT … FROM … WHERE … GROUP BY … HAVING … ORDER BY … LIMIT`; the database **conceptually evaluates** it starting with `FROM`. This *logical query processing order* is defined by the SQL standard's semantics: each step takes the previous step's table and hands a new table to the next, so each step only sees what earlier steps produced.

Step	Clause	Input → output	Can refer to
1	`FROM`, `JOIN … ON`	tables → one combined row set	table columns
2	`WHERE`	rows → fewer rows	table columns (no aggregates, no `SELECT` aliases)
3–4	`GROUP BY` + aggregates	rows → one row per group	table columns / expressions
5	`HAVING`	groups → fewer groups	grouping columns, aggregates
6	`SELECT`	groups/rows → output columns	grouping columns, aggregates; **window functions run here**
7	`DISTINCT`	output rows → unique output rows	output columns
8	set operators (`UNION`, `INTERSECT`, `EXCEPT`; Chapter 6)	two results → one	output columns
9	`ORDER BY`	rows → sorted rows	output aliases, output positions, and (without `DISTINCT`/set ops) input columns
10	`LIMIT`/`OFFSET`	sorted rows → a slice	—

Why it matters

It explains almost every "column does not exist" or "aggregate not allowed" error:

- **`WHERE` can't use a `SELECT` alias** — the alias is created in step 6, `WHERE` is step 2.
- **`WHERE` can't use aggregates** — groups don't exist until step 3.
- **`ORDER BY` *can* use aliases** — it runs after `SELECT`.
- **`LIMIT` applies to the sorted, final rows** — so `ORDER BY … LIMIT 3` really means "top 3".
- **Window functions can't be used in `WHERE`** — they are computed in step 6 (Chapter 8).

It also guides efficiency and debugging: filter early (`WHERE`, step 2) so fewer rows are grouped and sorted; and trace a surprising result step by step — what `FROM` produces, what survives `WHERE`, what groups form.

```
SELECT emp_id, salary * 12 AS yearly FROM employees WHERE yearly > 2000000;
```

Output:

```
ERROR:  column "yearly" does not exist
LINE 1: ...mp_id, salary * 12 AS yearly FROM employees WHERE yearly > 2...
                                                             ^
```

```
SELECT emp_id, salary * 12 AS yearly FROM employees ORDER BY yearly DESC LIMIT 3;
```

Output:

```
 emp_id |   yearly
--------+------------
      1 | 3000000.00
      2 | 2220000.00
      6 | 1800000.00
(3 rows)
```

Fix the `WHERE` case by repeating the expression (`WHERE salary * 12 > 2000000`) or computing it in a **derived table** (a subquery in `FROM`, Chapter 6) so it becomes an input column:

```
SELECT * FROM (SELECT emp_id, salary * 12 AS yearly FROM employees) t
WHERE yearly > 2000000;
```

Output:

```
 emp_id |   yearly
--------+------------
      1 | 3000000.00
      2 | 2220000.00
(2 rows)
```

A query that uses every clause

```sql
SELECT   dept_id, count(*) AS headcount, round(avg(salary)) AS avg_salary
FROM     employees
WHERE    hire_date >= '2018-01-01'
GROUP BY dept_id
HAVING   count(*) >= 2
ORDER BY avg_salary DESC
LIMIT    3;
```

Output:

```
 dept_id | headcount | avg_salary
---------+-----------+------------
       1 |         4 |     130750
       3 |         2 |      90500
       2 |         3 |      69333
(3 rows)
```

Trace: **FROM** 15 rows → **WHERE** drops Asha (2015), Kiran (2016), Daniel (2017) → 12 rows → **GROUP BY** groups 1 (4), 2 (3), 3 (2), 4 (1: Omar), 5 (1: Li), `NULL` (1: Rohan) → aggregates → **HAVING** keeps 1, 2, 3 → **SELECT** names `headcount`, `avg_salary` → **ORDER BY** the alias → **LIMIT 3** keeps all three.

`DISTINCT` happens after `SELECT` and before `ORDER BY`/`LIMIT`, and `ORDER BY` may sort by an aggregate that is not selected:

```sql
SELECT DISTINCT job_title FROM employees ORDER BY job_title LIMIT 4;

SELECT dept_id FROM employees WHERE dept_id IS NOT NULL
GROUP BY dept_id ORDER BY sum(salary) DESC;
```

Output:

```
   job_title
----------------
 Accountant
 CEO
 Content Writer
 Engineer
(4 rows)

 dept_id
---------
       1
       2
       4
       3
       5
(5 rows)
```

Logical order is not physical order

The logical order defines **what the answer is**, not **how it is computed**. The planner may push a `WHERE` into a join, read rows pre-sorted from an index and skip the sort, stop once `LIMIT` rows are found, or move a `HAVING` on a grouping column into the scan. `EXPLAIN` shows the physical plan (Chapter 12).

Dialect notes

Behaviour	PostgreSQL	MySQL	SQLite	SQL Server
`SELECT` alias in `GROUP BY`	allowed (extension)	allowed	allowed	**not** allowed
`SELECT` alias in `HAVING`	not allowed	allowed	allowed	not allowed
`SELECT` alias in `WHERE`	not allowed	not allowed	allowed (quirk)	not allowed
Row limit	`LIMIT n OFFSET m` / `FETCH FIRST n ROWS ONLY`	`LIMIT`	`LIMIT`	`TOP n` / `OFFSET … FETCH`

The extensions work because those databases look the name up among the output columns; they do not change the logical order.

In the interview

> **30-second answer.** "Logically, SQL runs `FROM`/joins, then `WHERE`, `GROUP BY`, aggregates, `HAVING`, `SELECT` (including window functions), `DISTINCT`, set operators, `ORDER BY`, and finally `LIMIT`/`OFFSET`. That's why `WHERE` can't see aliases or aggregates but `ORDER BY` can. The optimizer may physically reorder work as long as the result is the same."

- *Q: Why can't I filter on `ROW_NUMBER()` in `WHERE`?* — Window functions are computed in the `SELECT` step, after `WHERE`. Wrap the query in a subquery or CTE (Common Table Expression — a named subquery written with `WITH`, Chapter 7) and filter outside.
- *Q: Is `LIMIT` applied before or after `ORDER BY`?* — After. Without `ORDER BY`, which rows you get is undefined.
- *Q: Does `WHERE` run before the join?* — Logically after `FROM` builds the joined rows; physically the planner often filters each table before joining.

Self-check

1. Order: `HAVING`, `FROM`, `ORDER BY`, `SELECT`, `WHERE`, `GROUP BY`, `LIMIT`? → `FROM`, `WHERE`, `GROUP BY`, `HAVING`, `SELECT`, `ORDER BY`, `LIMIT` (mnemonic F-W-G-H-S-D-O-L, with `DISTINCT` after `SELECT`).
2. Where do window functions run? → In the `SELECT` step — after `HAVING`, before `DISTINCT`/`ORDER BY`.

4.6 Subtotals: GROUPING SETS, ROLLUP and CUBE

What it is

Reports often need **several levels at once**: revenue per country *and* category, per-country subtotals, a grand total. `GROUPING SETS`, `ROLLUP` and `CUBE` (SQL:1999; PostgreSQL 9.5+) produce them in one query, as if several `GROUP BY` queries were stacked with `UNION ALL` (which appends the rows of several queries).

Syntax	Groupings produced	Example with `(country, category)`
`GROUPING SETS ((a, b), (a), ())`	exactly the ones you list	whatever you list
`ROLLUP (a, b)`	`(a, b)`, `(a)`, `()` — a hierarchy, dropping columns from the right	detail, per-country subtotal, grand total
`CUBE (a, b)`	every subset: `(a, b)`, `(a)`, `(b)`, `()` — 2^n groupings	detail, per-country, per-category, grand total

A "rolled-up" column shows as `NULL` in subtotal rows. `GROUPING(col)` returns 1 when `col` is `NULL` *because it was rolled up* and 0 otherwise — telling a subtotal `NULL` from a genuine data `NULL`.

How it works

A view (a saved query, Chapter 7) with one row per non-cancelled order line:

```
CREATE VIEW sales AS
SELECT c.country, coalesce(p.category, 'Other') AS category,
       oi.quantity * oi.unit_price * (1 - oi.discount) AS revenue
FROM order_items oi
JOIN orders o    ON o.order_id = oi.order_id
JOIN customers c ON c.customer_id = o.customer_id
JOIN products p  ON p.product_id = oi.product_id
WHERE o.status <> 'cancelled';
```

ROLLUP — detail, then a subtotal per country, then the grand total:

```
SELECT country, category, round(sum(revenue), 2) AS revenue
FROM sales
GROUP BY ROLLUP (country, category)
ORDER BY country NULLS LAST, category NULLS LAST;
```

Output:

```
 country |  category   | revenue
---------+-------------+---------
 India   | Accessories |  621.85
 India   | Electronics | 2558.04
 India   | Furniture   | 1441.47
 India   | (null)      | 4621.36
 UK      | Accessories |   49.50
 UK      | Electronics | 1808.86
 UK      | Furniture   |  499.00
 UK      | (null)      | 2357.36
 USA     | Electronics |  324.90
 USA     | Other       |  100.00
 USA     | (null)      |  424.90
 (null)  | (null)      | 7403.62
(12 rows)
```

`category = (null)` rows are per-country subtotals; the last row is the grand total. Labelling them with `GROUPING()`:

```
SELECT CASE WHEN grouping(country) = 1 THEN 'ALL countries' ELSE country END AS country,
       CASE WHEN grouping(category) = 1 THEN 'ALL categories' ELSE category END AS category,
       round(sum(revenue), 2) AS revenue
FROM sales
GROUP BY ROLLUP (country, category)
ORDER BY grouping(country), country, grouping(category), category;
```

Output:

```
    country    |    category    | revenue
---------------+----------------+---------
 India         | Accessories    |  621.85
 ...
 India         | ALL categories | 4621.36
 ...
 USA           | ALL categories |  424.90
 ALL countries | ALL categories | 7403.62
(12 rows)
```

CUBE adds per-category totals across countries. `grouping(country, category)` returns a bit mask (each binary digit flags one column: left bit `country`, right bit `category`) — 0 detail, 1 `category` rolled up, 2 `country` rolled up, 3 grand total:

```
SELECT country, category, round(sum(revenue), 2) AS revenue,
       grouping(country, category) AS level_bits
FROM sales
GROUP BY CUBE (country, category)
ORDER BY level_bits, country, category;
```

Output:

```
 country |  category   | revenue | level_bits
---------+-------------+---------+------------
 India   | Accessories |  621.85 |          0
 ...                               (8 detail rows)
 India   | (null)      | 4621.36 |          1
 UK      | (null)      | 2357.36 |          1
 USA     | (null)      |  424.90 |          1
 (null)  | Accessories |  671.35 |          2
 (null)  | Electronics | 4691.80 |          2
 (null)  | Furniture   | 1940.47 |          2
 (null)  | Other       |  100.00 |          2
 (null)  | (null)      | 7403.62 |          3
(16 rows)
```

GROUPING SETS picks exactly the levels — per country, per category, grand total, no detail:

```
SELECT country, category, count(*) AS lines
FROM sales
GROUP BY GROUPING SETS ((country), (category), ())
ORDER BY country NULLS LAST, category NULLS LAST;
```

Output:

```
 country |  category   | lines
---------+-------------+-------
 India   | (null)      |    11
 UK      | (null)      |     5
 USA     | (null)      |     4
 (null)  | Accessories |     6
 (null)  | Electronics |     9
 (null)  | Furniture   |     4
 (null)  | Other       |     1
 (null)  | (null)      |    20
(8 rows)
```

Trade-offs

Option	Pros	Cons
`ROLLUP` / `CUBE` / `GROUPING SETS`	One statement, one scan of the input, totals always consistent	Output mixes levels — readers must understand the `NULL`s; `CUBE` grows as 2^n
Several `GROUP BY` queries + `UNION ALL`	Works in every database	Repeats the query; scans the input once per level
Totals in the application / BI tool (business-intelligence reporting software)	Flexible formatting	Moves more data out of the database

Database	Support
PostgreSQL 9.5+	`GROUPING SETS`, `ROLLUP`, `CUBE`, `GROUPING()`
SQL Server, Oracle	all of the above
MySQL 8	`GROUP BY a, b WITH ROLLUP` and `GROUPING()`; no `CUBE` or `GROUPING SETS`
SQLite	none — use `UNION ALL`

Common mistakes

- Confusing a subtotal `NULL` with a real one (use `GROUPING()`, or `COALESCE` real `NULL`s to a label first, as the view does).
- Expecting subtotals in some order — `ORDER BY` (often on `GROUPING(...)`).
- `CUBE` on many columns: 5 columns = 32 groupings.

In the interview

> **30-second answer.** "`ROLLUP(a, b)` gives `(a, b)`, `(a)` and the grand total — hierarchical subtotals. `CUBE(a, b)` gives every combination including `(b)` alone. `GROUPING SETS` lets you list the exact groupings. Rolled-up columns show as `NULL`, and `GROUPING(col)` tells those apart from real `NULL`s."

- *Q: How many groupings does `CUBE(a, b, c)` produce?* — $2^3 = 8$.
- *Q: How would you add a grand-total row without `ROLLUP`?* — `UNION ALL` a second query with no `GROUP BY`.

Self-check

1. Groupings of `ROLLUP (year, month, day)`? → `(year, month, day)`, `(year, month)`, `(year)`, `()`.
2. `GROUP BY ROLLUP (a)` as `GROUPING SETS`? → `GROUP BY GROUPING SETS ((a), ())`.

Chapter 4 TL;DR

Idea	Remember
Aggregates	`COUNT(*)` counts rows; every other aggregate (and `COUNT(col)`) skips `NULL`s. Empty input → 0 for `COUNT`, `NULL` for the rest.
`GROUP BY`	One output row per distinct key; select only keys and aggregates (or columns of a grouped primary key). `NULL`s form one group. No automatic ordering.
`HAVING`	Filters groups after aggregation; `WHERE` filters rows before. Aggregates only in `HAVING`.
Conditional aggregation	`agg(...) FILTER (WHERE …)` or `SUM(CASE …)` — many filtered numbers in one scan; ratios with `100.0 *`.
Logical order	FROM → WHERE → GROUP BY → aggregates → HAVING → SELECT (windows) → DISTINCT → set ops → ORDER BY → LIMIT.
Subtotals	`ROLLUP`, `CUBE`, `GROUPING SETS`; label rolled-up rows with `GROUPING()`.

Next: Chapter 5 — Joins combines rows from several tables, which is what most real aggregations are built on.